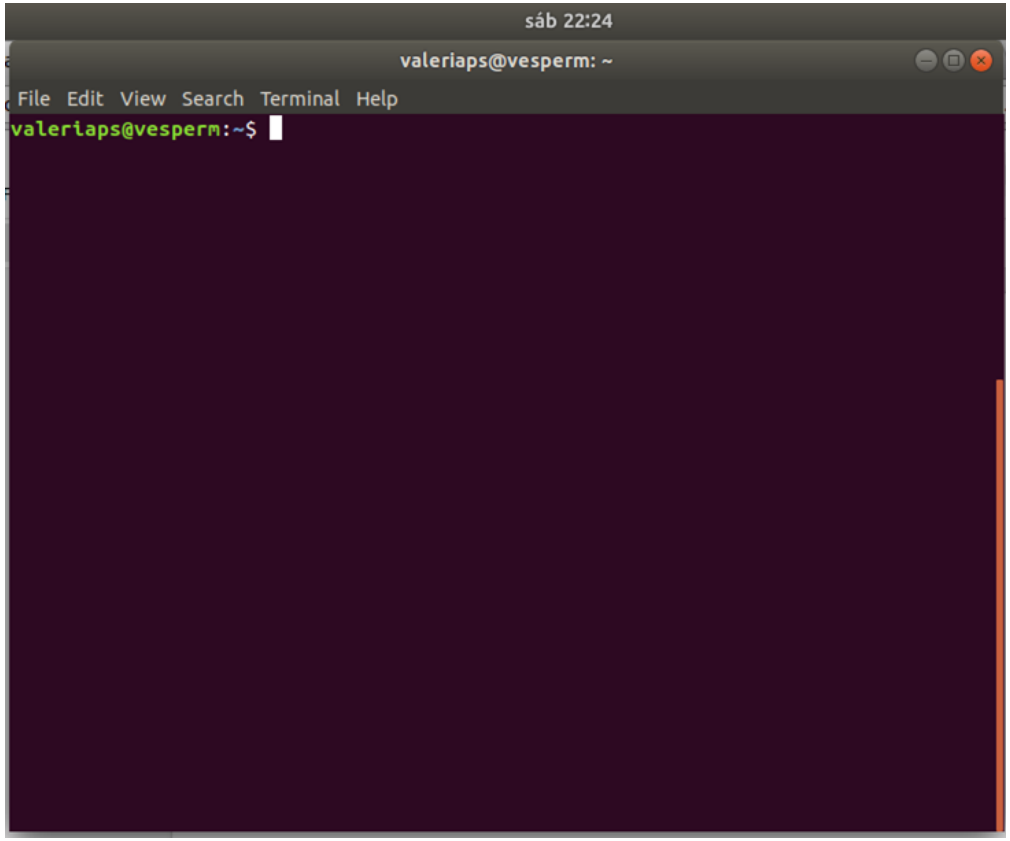
BASH

THE BOURNE-AGAIN SHELL

Tutorial - Compilando e executando pela linha de comando do Linux

O Bash é um interpretador de comandos que possibilita interagir com os aplicativos do seu computador por meio de comandos de texto.

Alguns aplicativos só são acessíveis por meio de comandos de texto. E isso vale tanto para sistemas Linux como para o Windows.



Porque comandos em texto?

Programas de computador nem sempre têm uma interface para o usuário interagir, ou seja, nem sempre terão uma GUI - Graphic User Interface. A interação visual facilita localizar opções.

Por outro lado, dar ordens ao computador por meio de comandos digitados é mais rápido e mais poderoso. Pode não parecer no começo, mas acredite: quando você aprender os comandos, vai perceber como é rápido.

Que coisas legais o Bash pode fazer?

TODO tipo de coisa legal. Por meio do Bash, podemos executar aplicativos, alterar configurações, compilar programas, obter informações de sistema, automatizar tarefas e até enviar emails.

É uma ferramenta muito poderosa! Vamos focar em compilar e executar programas

✯✯ Para ter uma idea, dentro do Bash, digite `apt-cache search .`

Instalando o Bash

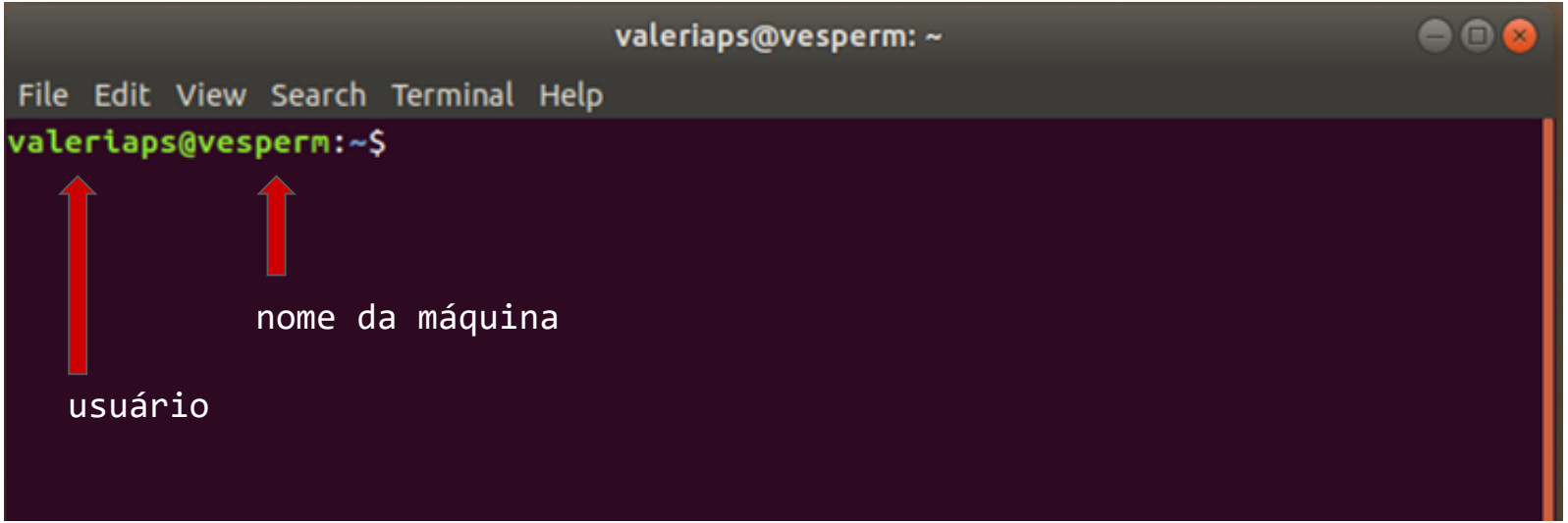
No Ubuntu, o Bash já vem instalado nativamente. No Windows, é possível instalá-lo por meio da loja do Windows seguindo as instruções disponíveis lá.

Utilizando pela primeira vez

Daqui pra frente, as instruções serão todas direcionadas para o sistema Ubuntu, que está instalado nos computadores Linux do DCC.

Vamos lá?

Abra o Bash (Ctrl + Alt + t)

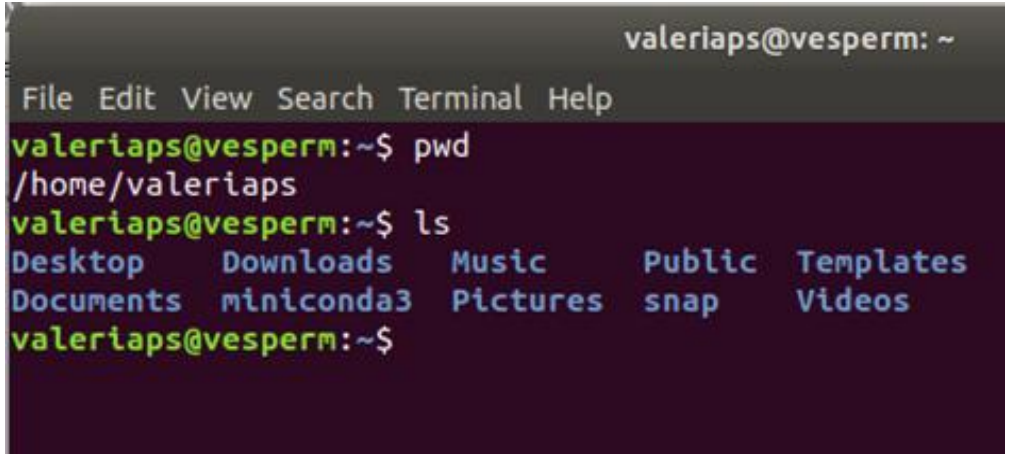


✯✯ É possível mudar o nome da máquina. Dê uma pesquisada sobre isso

Localize-se

Se você não diz para o Bash onde aplicar um determinado comando, ele vai realizá-lo no diretório atual e alguns comandos são diretório-específicos.

- Para saber em que diretório você está, dê o comando `pwd`.
- Para saber o conteúdo desta pasta, dê o comando `ls`



Navegando e criando diretórios

- Mude de diretório:
  `cd Documents`

- Crie um novo diretório:
  `mkdir novaPasta`

(Evite espaços no nome da sua pasta.)

```
valeriaps@vesperm: ~/Document
File  Edit  View  Search  Terminal  Help
valeriaps@vesperm:~$ pwd
/home/valeriaps
valeriaps@vesperm:~$ ls
Desktop    Downloads   Music     Public   Templates
Documents  miniconda3  Pictures  snap     Videos
valeriaps@vesperm:~$ cd Documents/
valeriaps@vesperm:~/Documents$ ls
valeriaps@vesperm:~/Documents$ mkdir meusCodigos
valeriaps@vesperm:~/Documents$ ls
meusCodigos
valeriaps@vesperm:~/Documents$
```

✯✯ A tecla TAB auto completa os nomes dos arquivos e diretórios, caso você tenha digitado caracteres suficientes

Instalando um compilador

Se você usou o CodeBlocks durante o curso, compilou utilizando o botão de compilar. Aquele botão chama um compilador que foi configurado no CodeBlocks. Geralmente é o GCC - *GNU Compiler Collection*.

- O GCC é instalado por padrão nos sistemas Unix. Verifique se o GCC está instalando, chamando a versão dele:

  `gcc -v`

```
valeriaps@vesperm:~/Documents$ gcc -v
Using built-in specs.
COLLECT_GCC=gcc
COLLECT_LTO_WRAPPER=/usr/lib/gcc/x86_64-linux-gnu/7/lto-wrapper
OFFLOAD_TARGET_NAMES=nvptx-none
OFFLOAD_TARGET_DEFAULT=1
Target: x86_64-linux-gnu
Configured with: ../src/configure -v --with-pkgversion='Ubuntu 7.
ith-bugurl=file:///usr/share/doc/gcc-7/README.Bugs --enable-langu
rig,d,fortran,objc,obj-c++ --prefix=/usr --with-gcc-major-version
fix=-7 --program-prefix=x86_64-linux-gnu- --enable-shared --enabl
-libexecdir=/usr/lib --without-included-gettext --enable-threads=
/lib --enable-nls --with-sysroot=/ --enable-clocale=gnu --enable-
nable-libstdcxx-time=yes --with-default-libstdcxx-abi=new --enabl
 --disable-vtable-verify --enable-libmpx --enable-plugin --enable
-system-zlib --with-target-system-zlib --enable-objc-gc=auto --en
sable-werror --with-arch-32=i686 --with-abi=m64 --with-multilib-l
enable-multilib --with-tune=generic --enable-offload-targets=nvpt
da-driver --enable-checking=release --build=x86_64-linux-gnu --ho
 --target=x86_64-linux-gnu
Thread model: posix
gcc version 7.2.0 (Ubuntu 7.2.0-8ubuntu3.2)
valeriaps@vesperm:~/Documents$
```

Compilando seu primeiro código pela linha de comando

Coloque algum código fonte em C dentro da pasta criada. Você pode fazer isso pelo Files, gerenciador de arquivos do Ubuntu, que tem interface gráfica.

- Agora compile seu programa:
  `gcc main.c -o programa`.
  - main.c = nome do seu código fonte
  - programa = nome que você quer dar ao executável
- Rode o programa:
  `./programa`

Quanto tempo seu programa leva pra rodar

Nas disciplinas de programação, você será requisitado a documentar o seu código. Nessa documentação, você apresentará alguns testes que você fez no seu programa, como por exemplo, quanto tempo ele levou para rodar

- Rode o programa verifique seu tempo de execução: `time ./programa`

✯✯ Seta para cima para buscar no histórico de comandos recentemente passados ao Bash

```
valeriaps@vesperm:~/Documents/meusCodigos$ gcc main.c -o pro
valeriaps@vesperm:~/Documents/meusCodigos$ ls
main.c  programa
valeriaps@vesperm:~/Documents/meusCodigos$ time ./programa
Eu sou um programa

real    0m0,002s
user    0m0,002s
sys     0m0,000s
valeriaps@vesperm:~/Documents/meusCodigos$
```

Opções de compilação

O GCC tem vários utilitários opcionais para te dar mais informações sobre o seu programa enquanto o compila. Chamamos esses utilitários de “flags de compilação”

- Vamos aprender duas flags. A primeira é a `-Wall`, uma coleção de flags que mostram erros e avisos

```
valeriaps@vesperm:~/Documents/meusCodigos$ gcc -Wall main.c -o programa
main.c: In function ‘main’:
main.c:5:6: warning: unused variable ‘a’ [-Wunused-variable]
  int a;
      ^
valeriaps@vesperm:~/Documents/meusCodigos$
```

✯✯ São tantas opções, mas tantas, que vamos deixar um link para você explorá-las:https://gcc.gnu.org/onlinedocs/gcc/Invoking-GCC.html#Invoking-GCC - Warning optionssh

- A mensagem de aviso diz que a variável ‘a’ não está sendo utilizada. De fato:

```
#include<stdio.h>

int main (){

        int a; // está gastando espaço na memória inutilmente
        printf("Eu sou um programa\n");

        return 0;
}
```

- Se retirarmos essa variável, o gcc vai parar de reclamar

- **warnings** são avisos que não impedem de gerar o executável. Resolvê-los é uma opção sua, mas recomendamos sempre dar ouvidos ao gcc
- **errors** por outro lado, não permitirão que seu programa seja compilado. Resolva-os.
  - ** Existem muitos avisos e erros possíveis. Não teríamos como tratá-los aqui. Se você não entender o que quer dizer algum deles, é muito provável que alguém da internet já teve esse problema e alguém já postou a solução. Caso não consiga resolver, entre em contato com a equipe do Programação 101!